Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

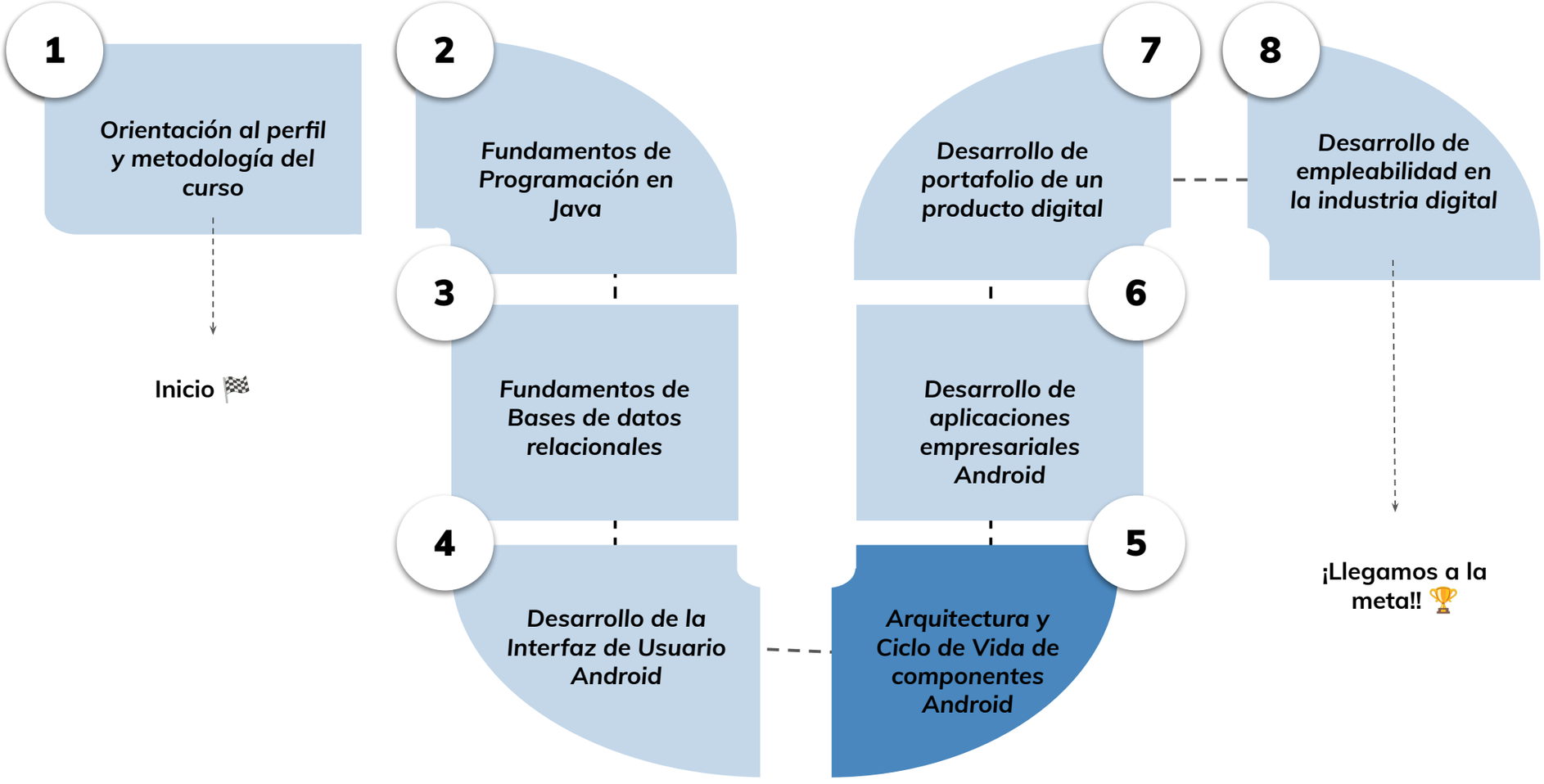
Ciclo de vida de componentes Android - Parte I

AE: Utilizar elementos del ciclo de vida para la implementación de un aplicativo android que resuelve un problema.

alkemy

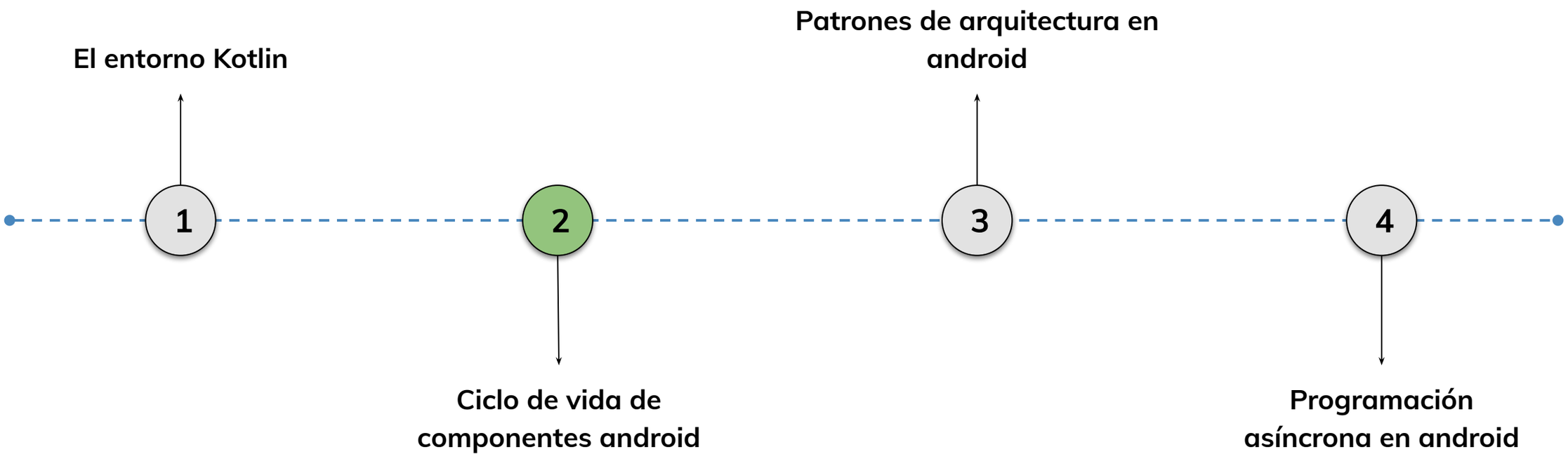
Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



Learning Path

¿Cuáles temas trabajaremos hoy?

5

.

Ciclo de Vida

- Android Life Cycle
- Ciclo de vida Actividades
- Permisos sensibles
- Utilizar el ciclo de vida
- Ciclo de vida de los fragmentos
- Los fragmentos y el método de factoría

¿Qué son?

Funciones

Usos

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- *Conocer Android Life Cycle*
- *Entender el ciclo de vida de las activities*
- *Comprender sus implicaciones*
- *Comprender la solicitud de permisos sensibles*
- *Utilizar el ciclo de vida a nuestro favor*
- *Conocer el ciclo de vida de los fragmentos*
- *Utilizar los fragmentos y el método de factoría*
- *Ver cómo aplicar estos comportamientos*

Rompehielo🥶

Contexto: 🙌

Tal vez hayan visto alguna vez a alguien que tiene muchas aplicaciones abiertas corriendo en su celular desde hace mucho tiempo pero no parece ralentizarse tanto como un computador lo haría con esa misma carga

Consigna: ✍️

¿Por qué creen que es el caso en dispositivos Android?

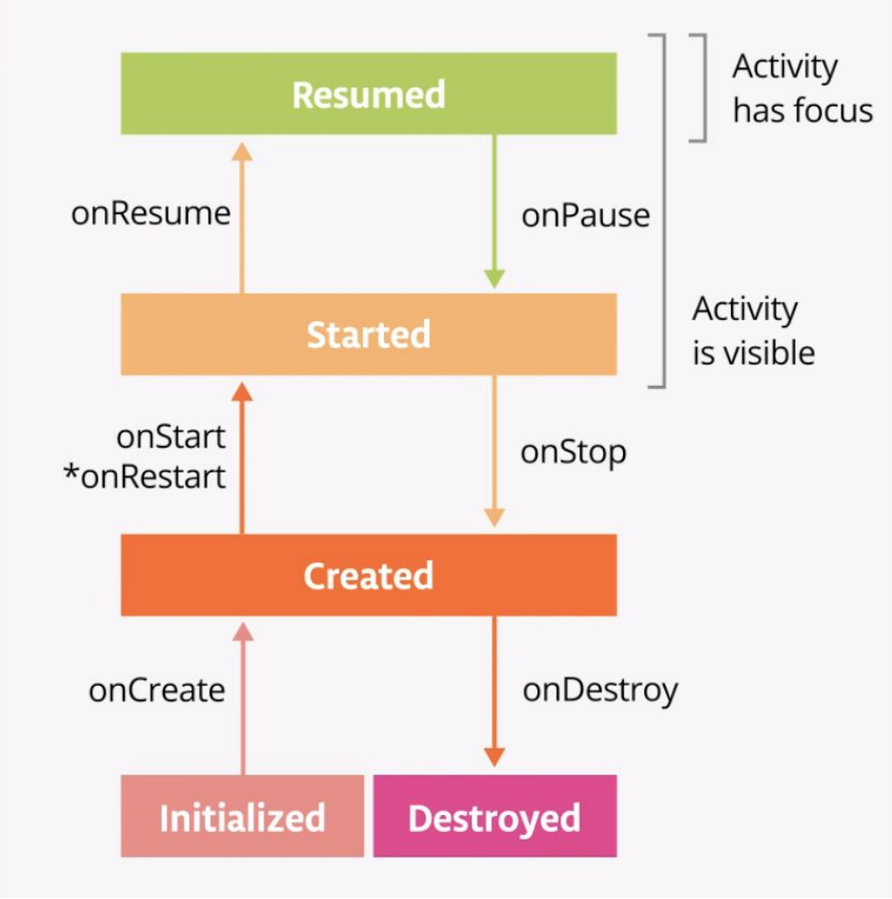
› Android Life Cycle

Android Life Cycle

¿Qué es?:

Son los estados y transiciones que experimenta una aplicación de Android desde el momento en que se inicia hasta que se detiene por completo. Comprender el ciclo de vida es esencial para desarrolladores de aplicaciones de Android, ya que les permite gestionar eficazmente los recursos y el comportamiento de la aplicación en diferentes situaciones.

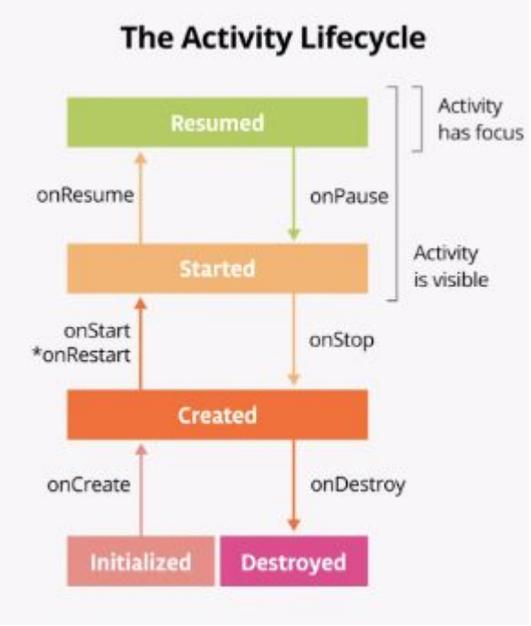
Android Life Cycle



Estados del ciclo de vida

¿Qué tenemos que tener en cuenta?:

Básicamente nos interesa si nuestra aplicación está en foco actual del usuario o si está corriendo de fondo/segundo plano. El sistema operativo nos pide que seamos responsables con los recursos que usamos entonces tenemos instancias en las cuales nos requiere librar dichos recursos y estar preparados para pausar e incluso terminar la ejecución de nuestra aplicaciones



› Ciclo de vida Actividades

Ciclo de vida Actividades

¿Qué es?:

El ciclo de vida de las actividades en Android se refiere al conjunto de estados por los que pasa una actividad (una pantalla o ventana de la aplicación) desde su creación hasta su destrucción. Todas las actividades de nuestra aplicaciones tienen que pasar por este ciclo para llegar a los estados que requiere siendo el de Running (Corriendo) el que denomina que nuestra actividad se encuentra activa actualmente.

Estados del ciclo de vida

Inicio de la Aplicación (Created)

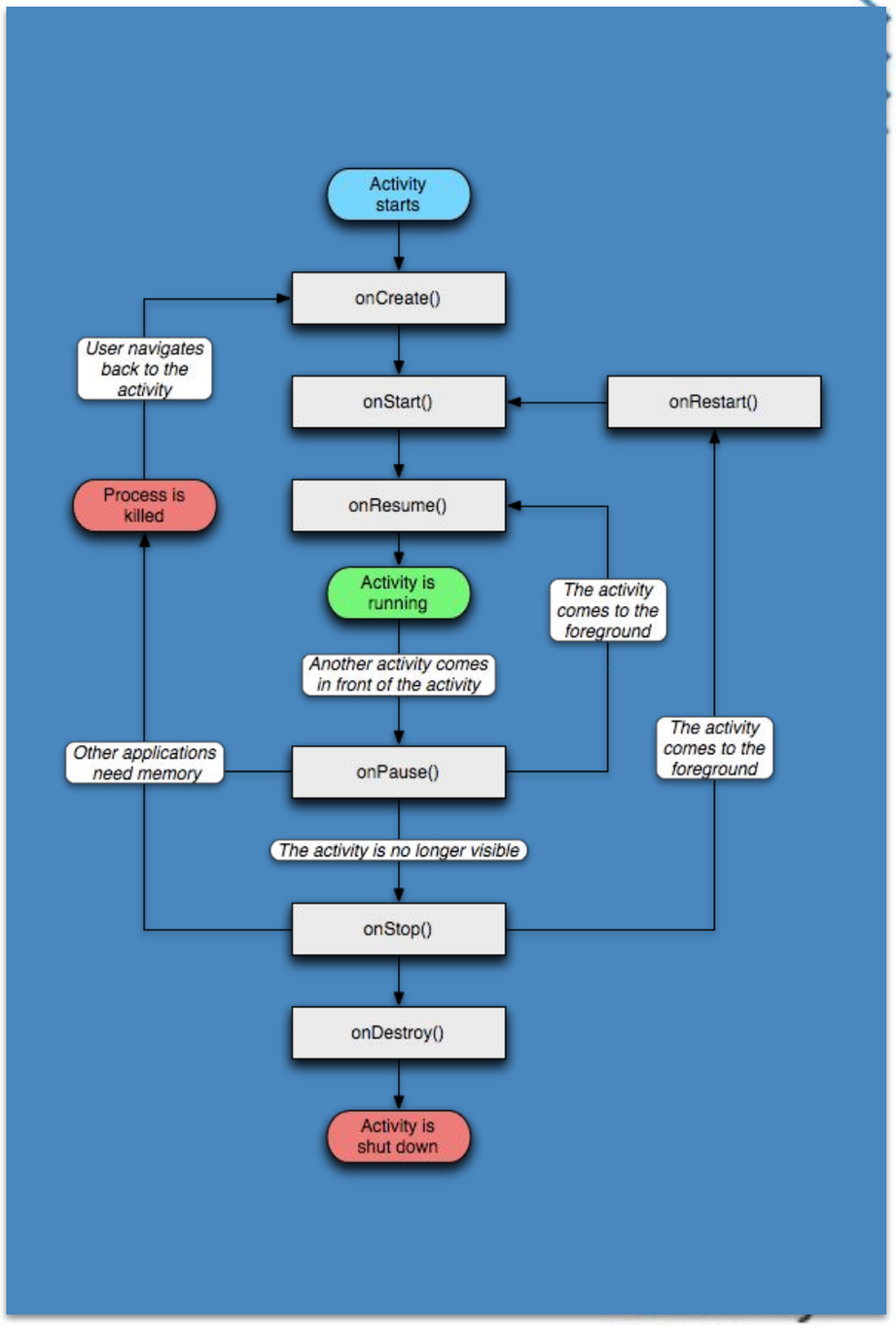
En Ejecución (Running)

En Pausa (Paused)

En Segundo Plano (Background)

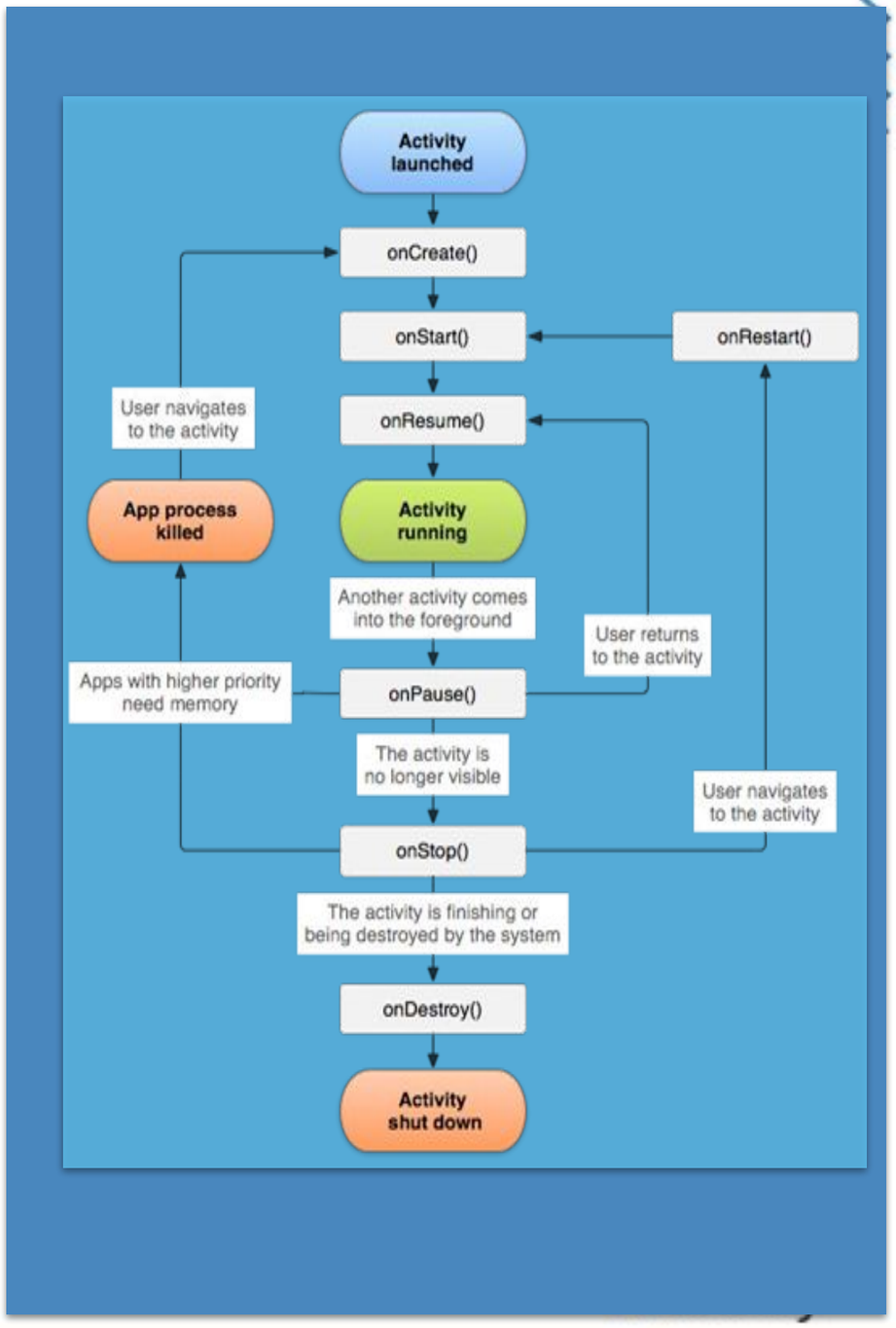
Detenida (Stopped)

Finalizada (Destroyed)



OnCreate

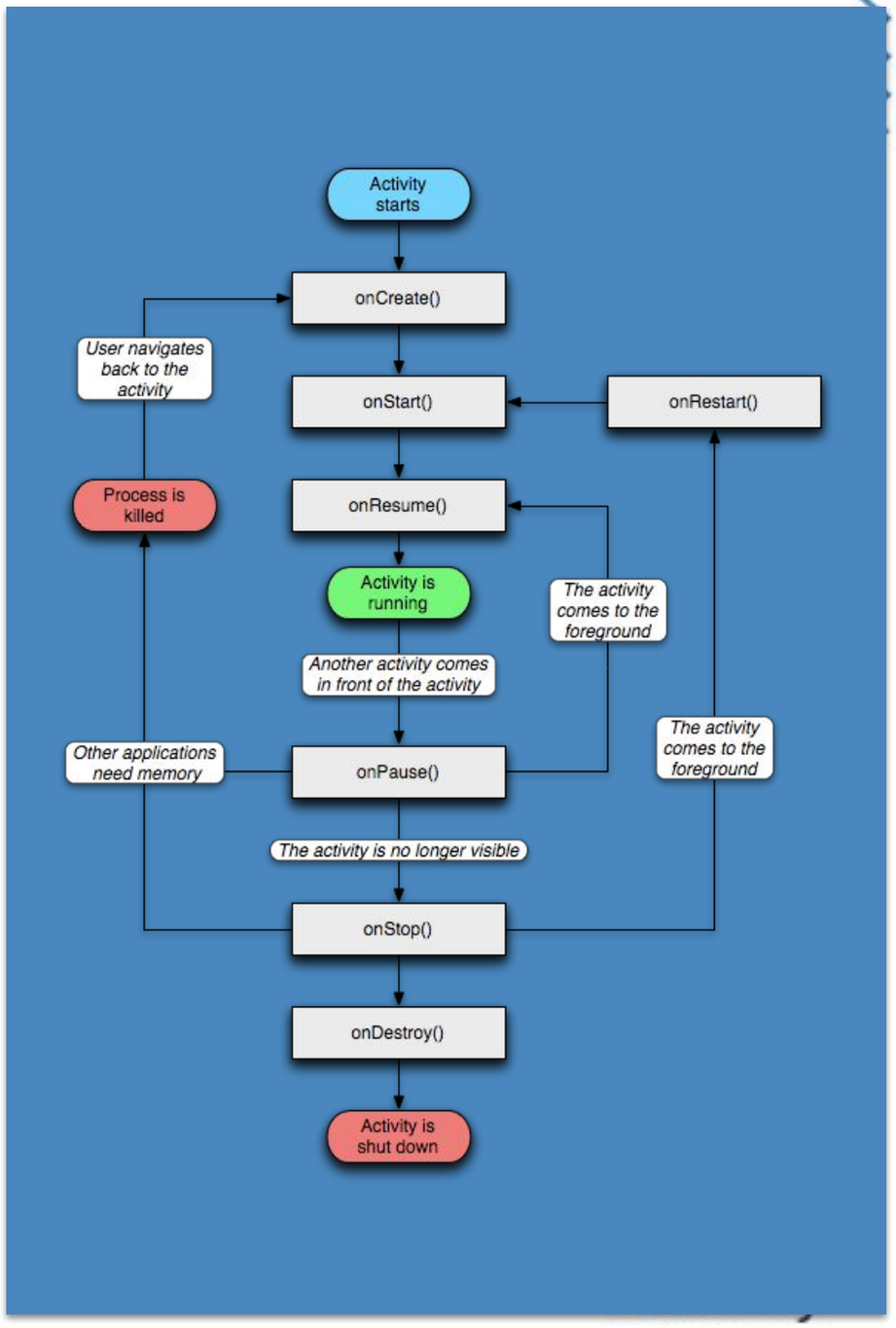
- El sistema crea la actividad
- Se ejecuta solo una vez en el ciclo de vida
- Se debe inicializar los componentes esenciales de la actividad
- Crear vistas
- Vincular datos a listas
- setContentView() para enlazar el diseño de la UI



OnStart

Preparativos finales para pasar a primer plano y convertirse en **interactiva**

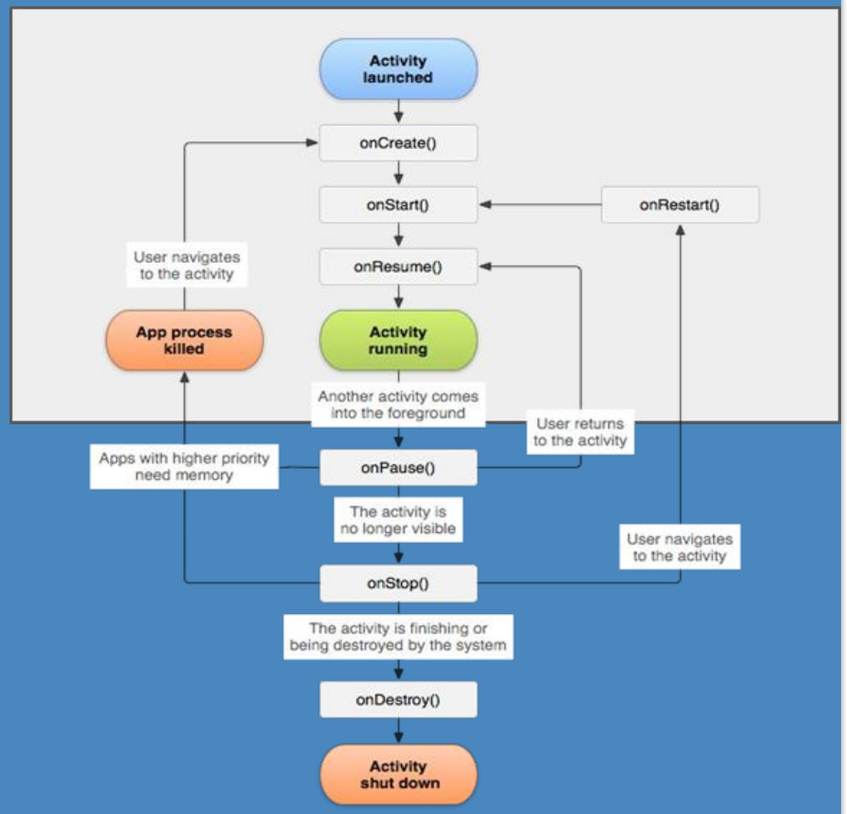
La app está en estado Iniciada y es visible para el usuario



OnResume

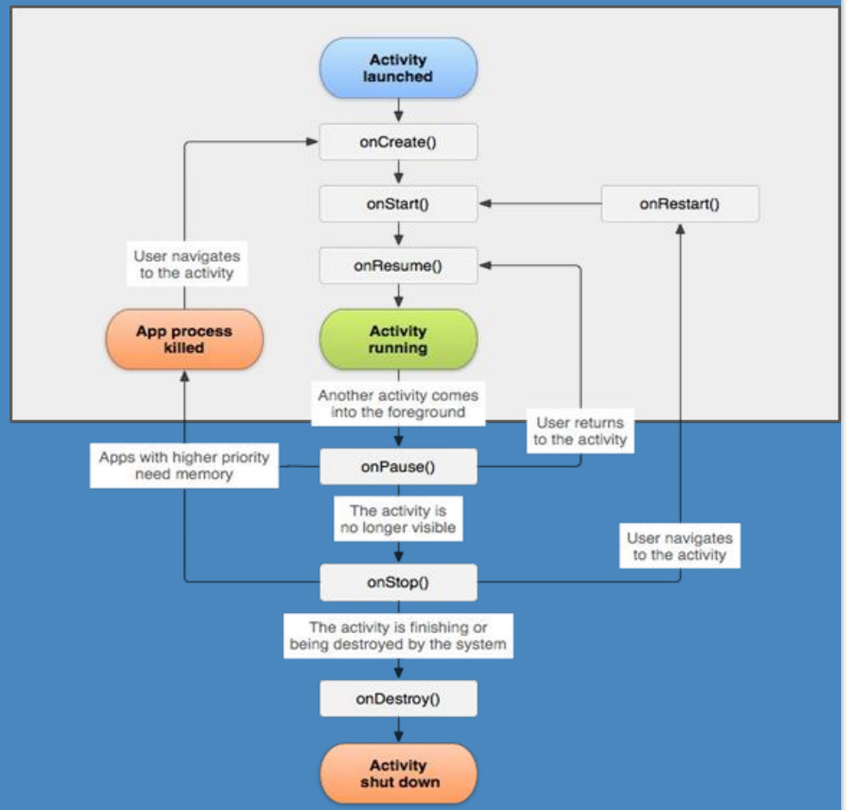
Se invoca justo antes de que la actividad comience a interactuar con el usuario

- La actividad es la primera en la pila de actividades
- Captura todo lo que el usuario ingresa



OnResume

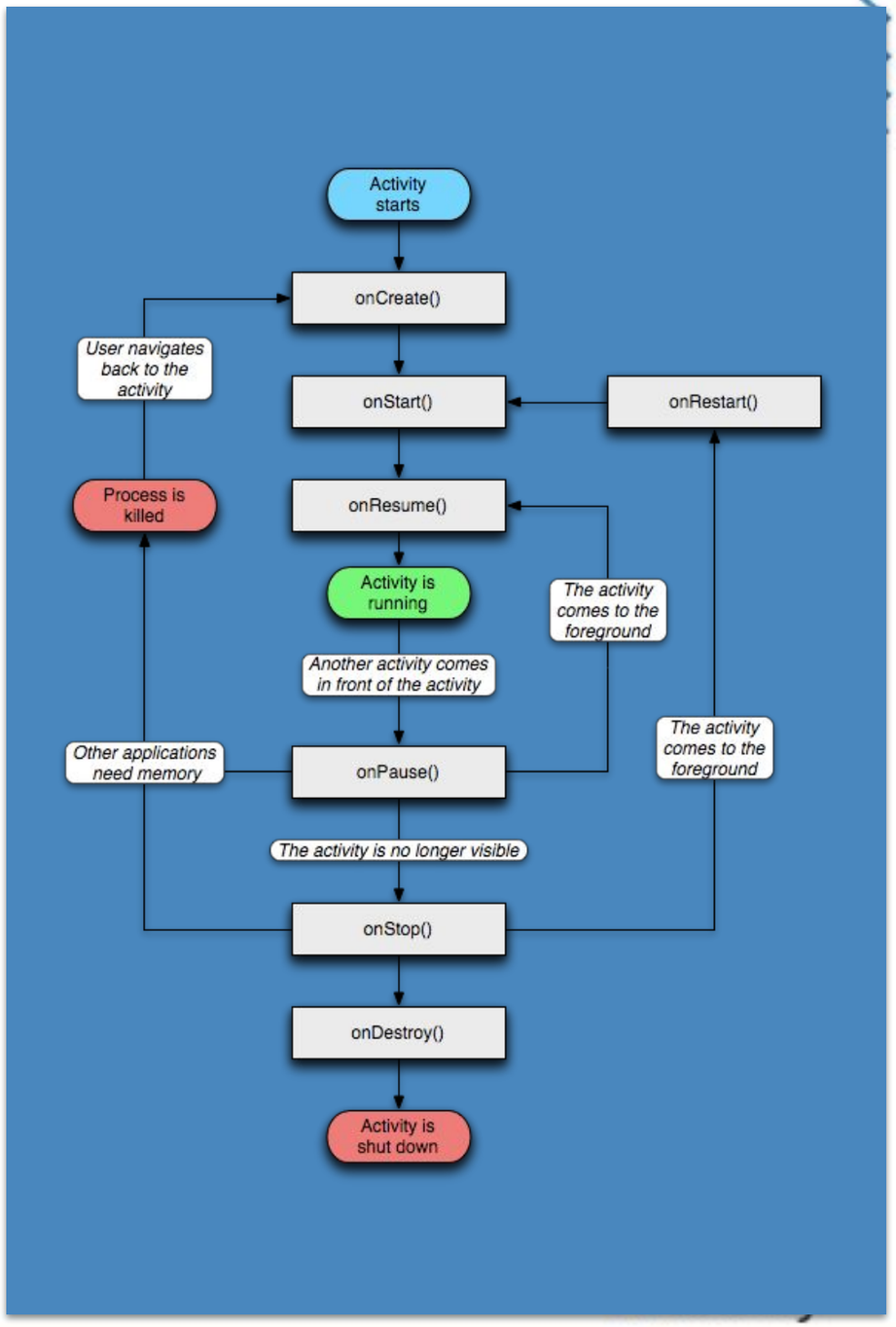
En este punto, la actividad es visible para el usuario y puede interactuar con la app



OnPause

La actividad pierde el foco y es parcialmente visible al usuario

- NO se deben guardar datos de la app, hacer llamadas de red o ejecutar transacciones de base de datos
- La actividad puede pasar a estado Detenida o Reanudada



Estados del ciclo de vida

Inicio de la Aplicación (Created)

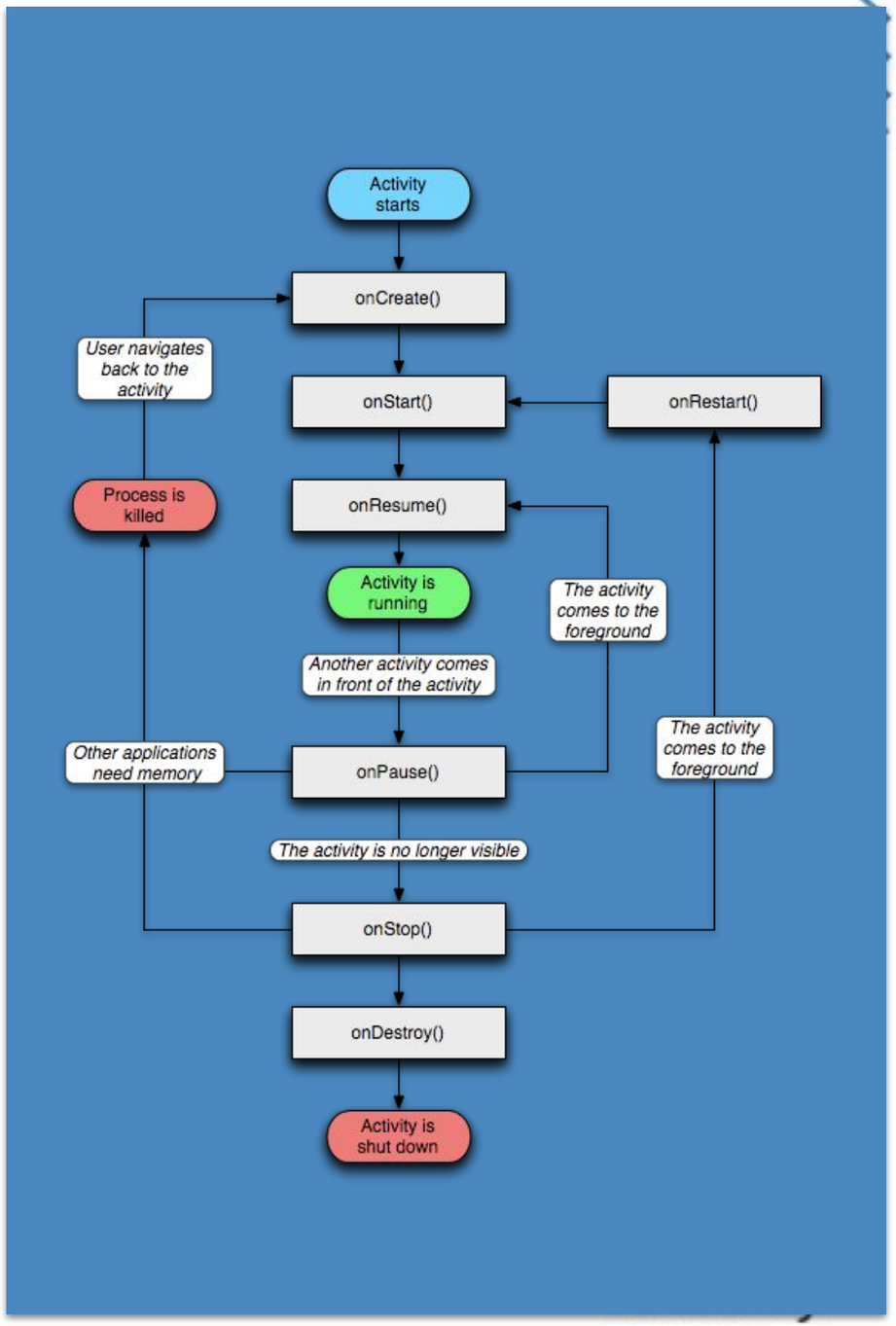
En Ejecución (Running)

En Pausa (Paused)

En Segundo Plano (Background)

Detenida (Stopped)

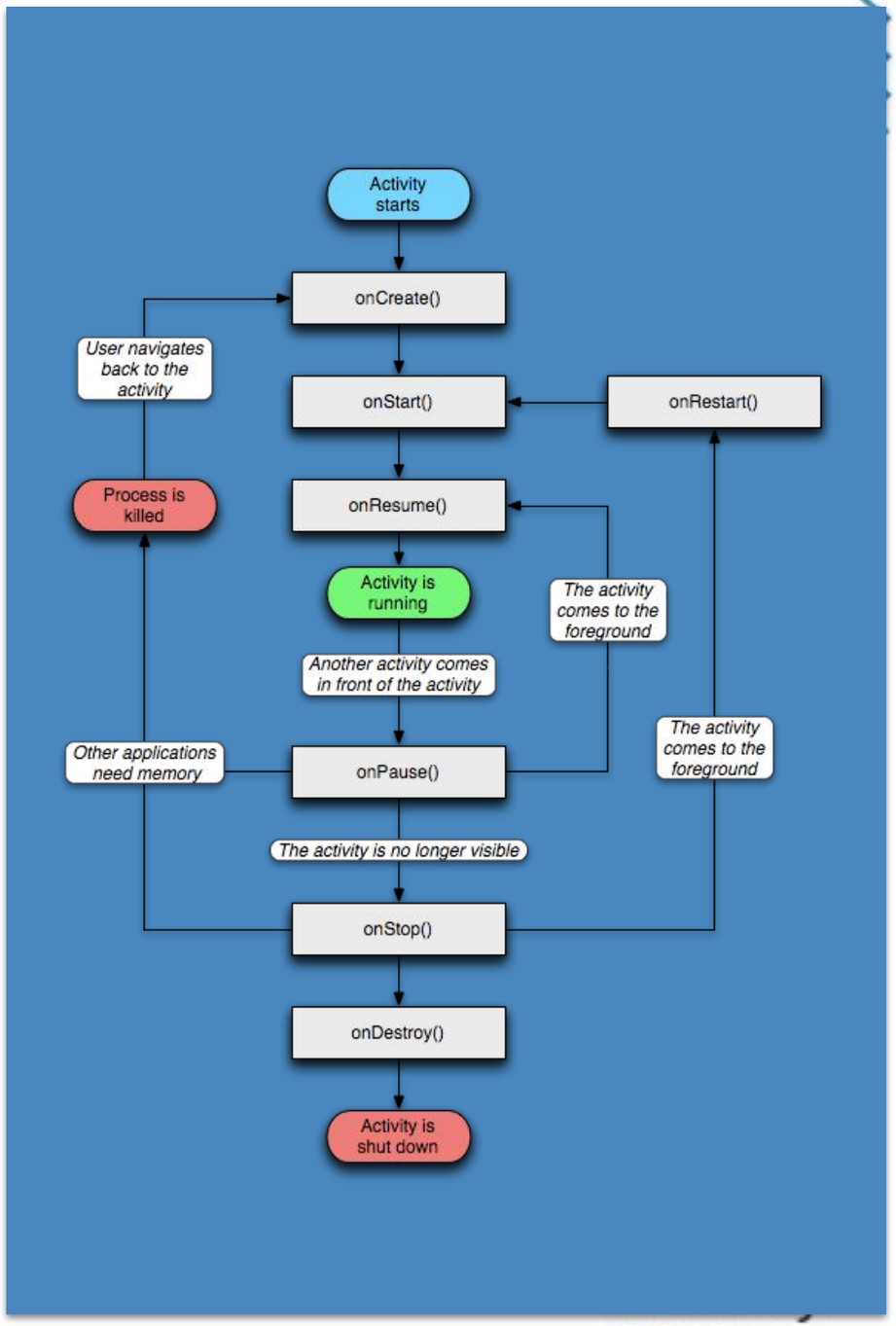
Finalizada (Destroyed)



OnPaused

La actividad pierde el foco y es parcialmente visible al usuario

- NO se deben guardar datos de la app, hacer llamadas de red o ejecutar transacciones de base de datos
- La actividad puede pasar a estado Detenida o Reanudada

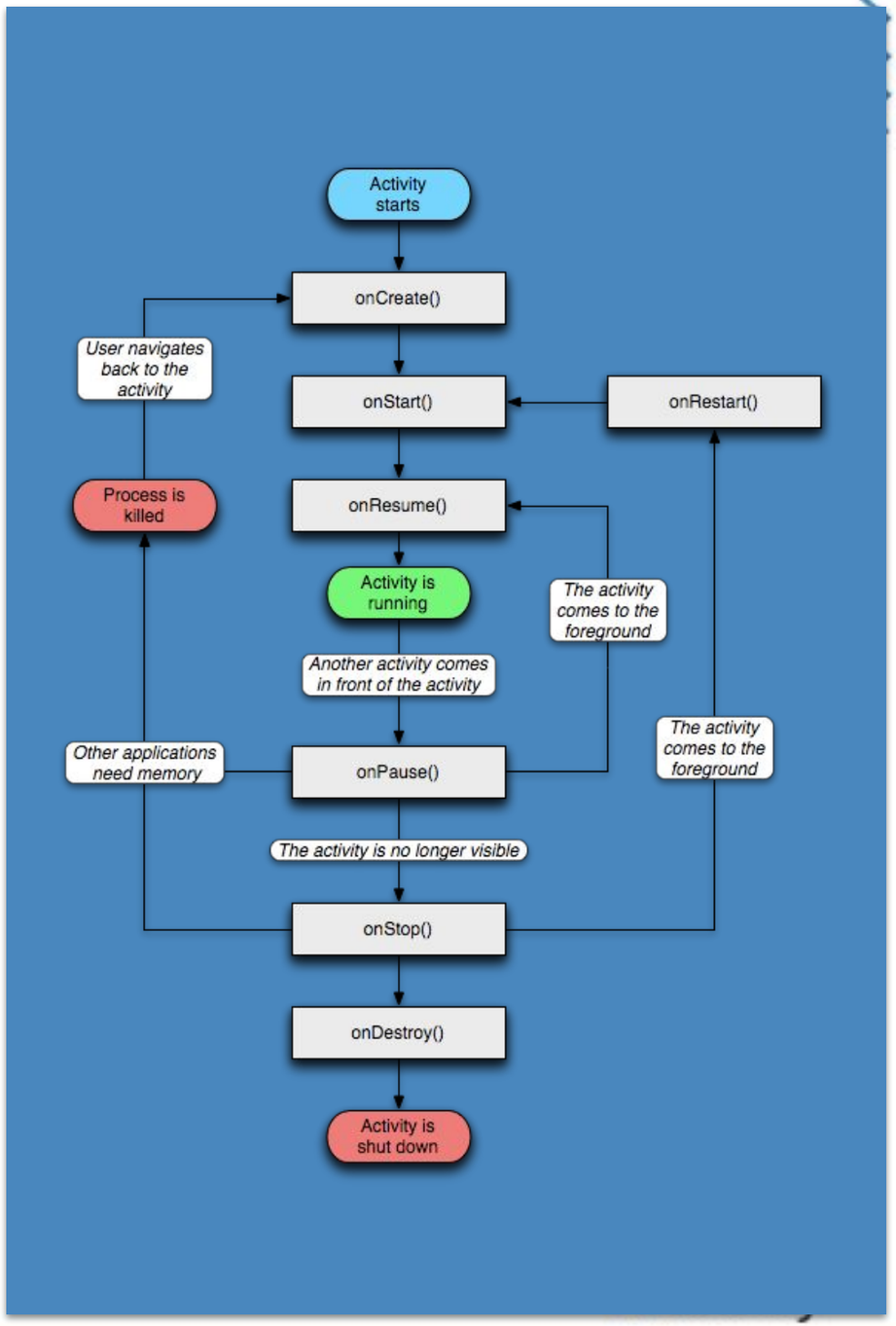


OnStop

La actividad ya no es visible para el usuario

- Se está eliminando la actividad
- Se inicia una nueva
- Ya existe una actividad que pasa a Reanudada y cubre la que se detuvo

Después de onStop() se puede ejecutar **onRestart()** o **onDestroy()**

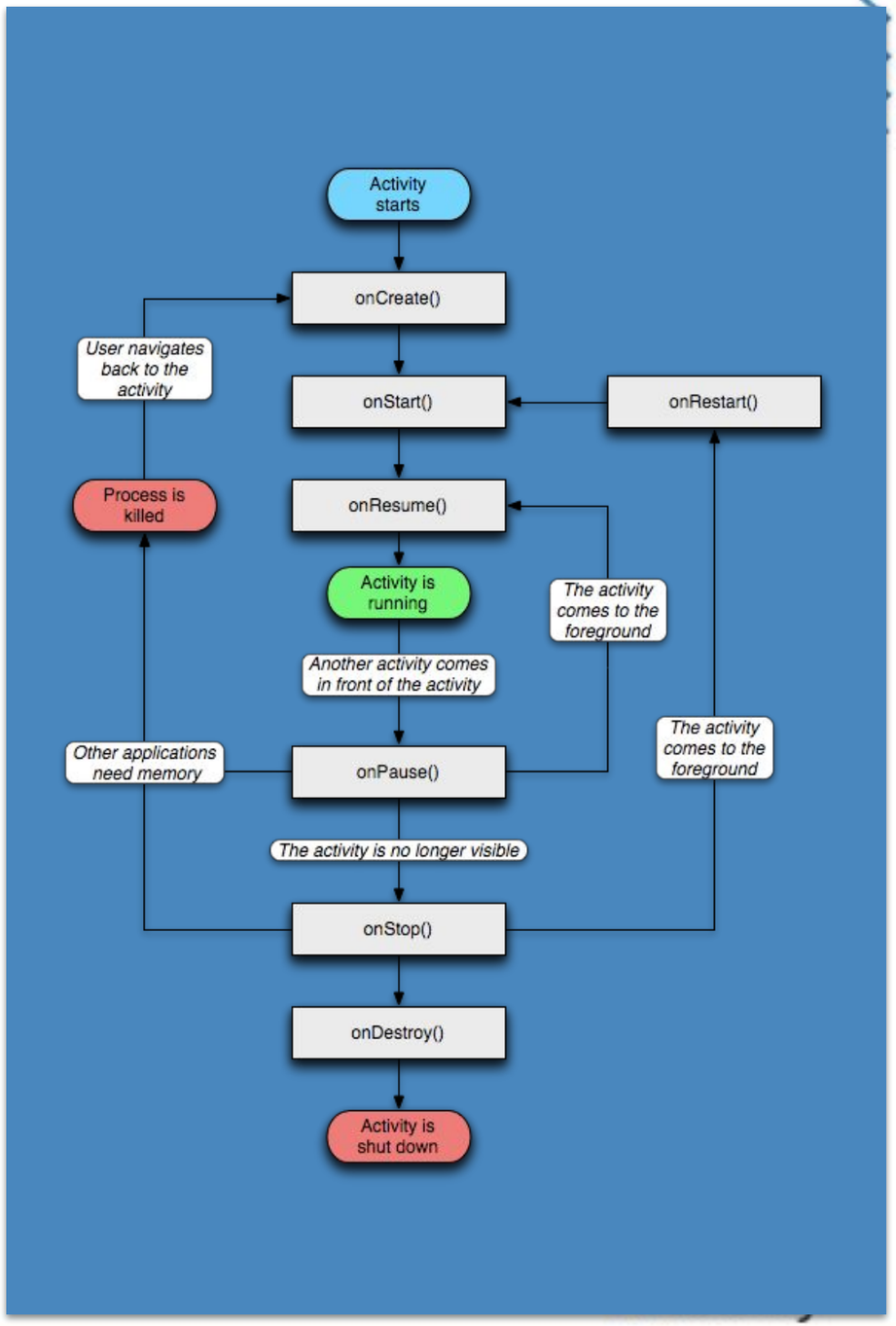


OnRestart

Una actividad Detenida está por volver a iniciarse

Restaura el estado de la actividad desde el momento en que se detuvo

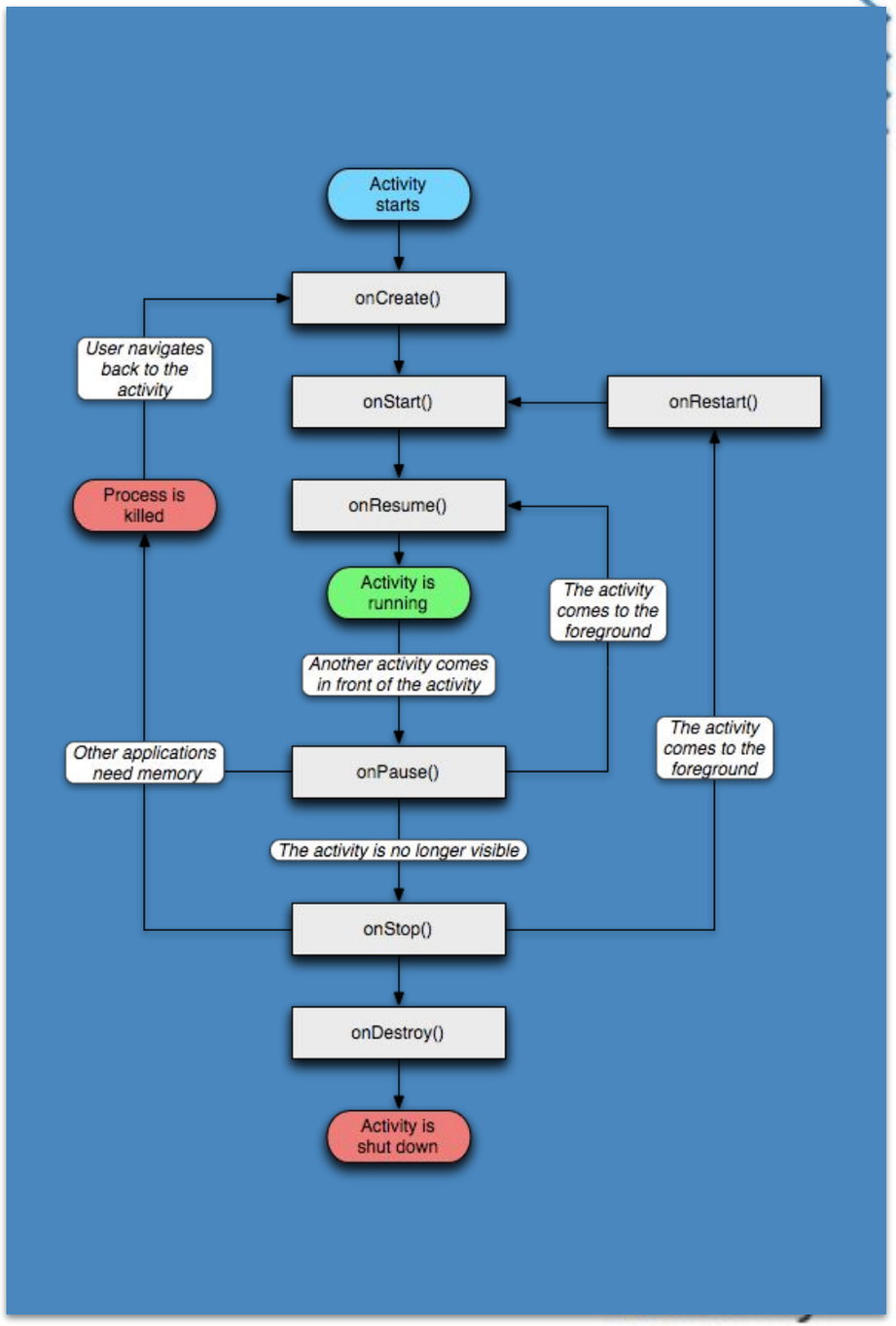
Luego siempre se ejecuta **onStart()**



OnDestroy

La actividad va a ser eliminada
Es la última llamada que recibe la actividad

Se utiliza para garantizar que todos los recursos sean liberados



LIVE CODING

Ejemplo en vivo

Uso del ciclo de vida:

Vamos a crear una aplicación para visualizar el ciclo de vida de la actividad

1. *Crear una actividad o aplicación nueva vacía en Kotlin*
2. *Dentro de la fun OnCreate agregar la linea Log.i("Ciclo de vida", "On Create")*
3. *Hacer override de todas las funciones del ciclo de vida*

4. *Agregar una entrada de Log.i("Ciclo de vida", <nombre fun>) donde se reemplaza <nombre fun> con el nombre de la función*

5. *Correr la aplicación y observar como minimizando la aplicación o cerrándola se reportan esos cambios en Logs*

Tiempo: 30 min

LIVE CODING SOLUCIÓN

Ejemplo en vivo

https://github.com/Mariocanedo/Ciclo_vida_Activitys

GitHub

¿Alguna consulta?

alkemy

Momento:

Time-out!

⏰5 -10 min.

alkemy

› Permisos sensibles

Permisos sensibles

¿Qué son?:

Son un conjunto de permisos de seguridad que implican un mayor nivel de acceso a datos o funciones del dispositivo. Estos permisos permiten que una aplicación realice acciones que pueden tener un impacto significativo en la privacidad y la seguridad del usuario. Los permisos sensibles son cuidadosamente regulados para proteger la información y el control del dispositivo del usuario.

Permisos sensible comunes

¿Cuáles son?:

Permiso de Ubicación (ACCESS_FINE_LOCATION)
Permiso de Cámara (CAMERA)
Permiso de Micrófono (RECORD_AUDIO)
Permiso de Contactos (READ_CONTACTS, WRITE_CONTACTS)
Permiso de SMS (SEND_SMS, READ_SMS)
Permiso de Llamadas (CALL_PHONE, READ_PHONE_STATE)

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Pedir sensible comunes

¿Cuáles son?:

En las versiones más modernas de Android tenemos que pedir al usuario explícitamente el usuario para poder usarlo Si el permiso no ha sido concedido, debes solicitarlo. Puedes mostrar un cuadro de diálogo explicando por qué necesitas el permiso y luego solicitarlo

LIVE CODING

Ejemplo en vivo

Permisos sensibles:

Agregar permisos de acceso a ubicación fina a una aplicación.

1. *Abrir o crear una aplicación*
2. *Abrir el manifiesto de la app*
3. *Agregar el permiso ACCESS_FINE_LOCATION*
4. *Correr la aplicación*
5. *Validar si se pide el permiso al iniciar la app*
6. *Hay que realizar pasos extra para tener el permiso?*

Tiempo: 20 min

› Utilizar el ciclo de vida

Utilizando el ciclo de vida a nuestro favor

¿Cómo hacerlo?:

El ciclo de vida de una actividad en Android puede ser aprovechado para optimizar el rendimiento, mejorar la experiencia del usuario y gestionar los recursos de manera eficiente.

Liberación de recursos

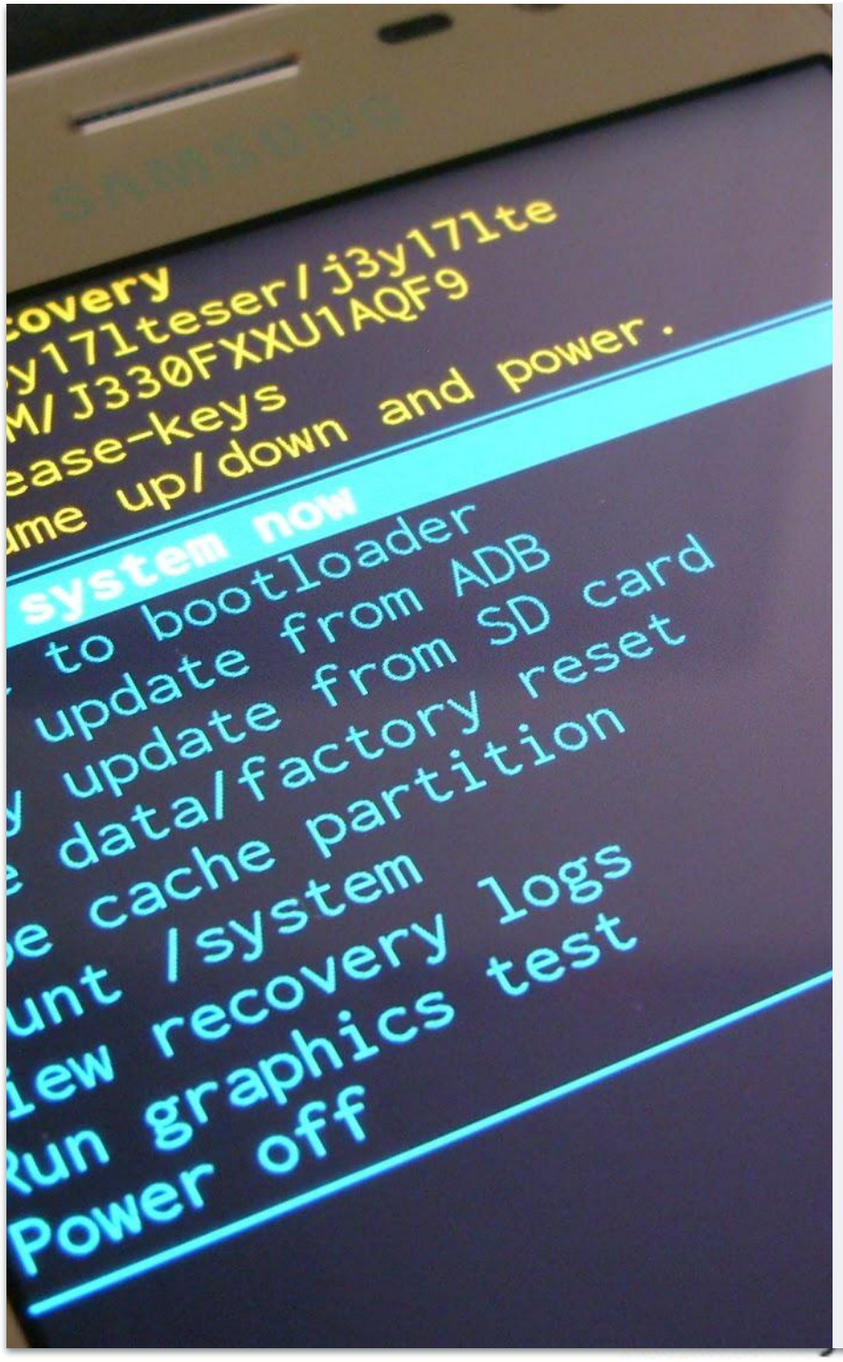
¿Cómo?:

Aprovecha los métodos onPause() y onStop() para liberar recursos que no son necesarios cuando la actividad está en segundo plano. Por ejemplo, puedes detener la reproducción de música, liberar conexiones a bases de datos o cerrar cualquier recurso que esté consumiendo memoria.

Guardado y restauración de datos

¿Cómo?:

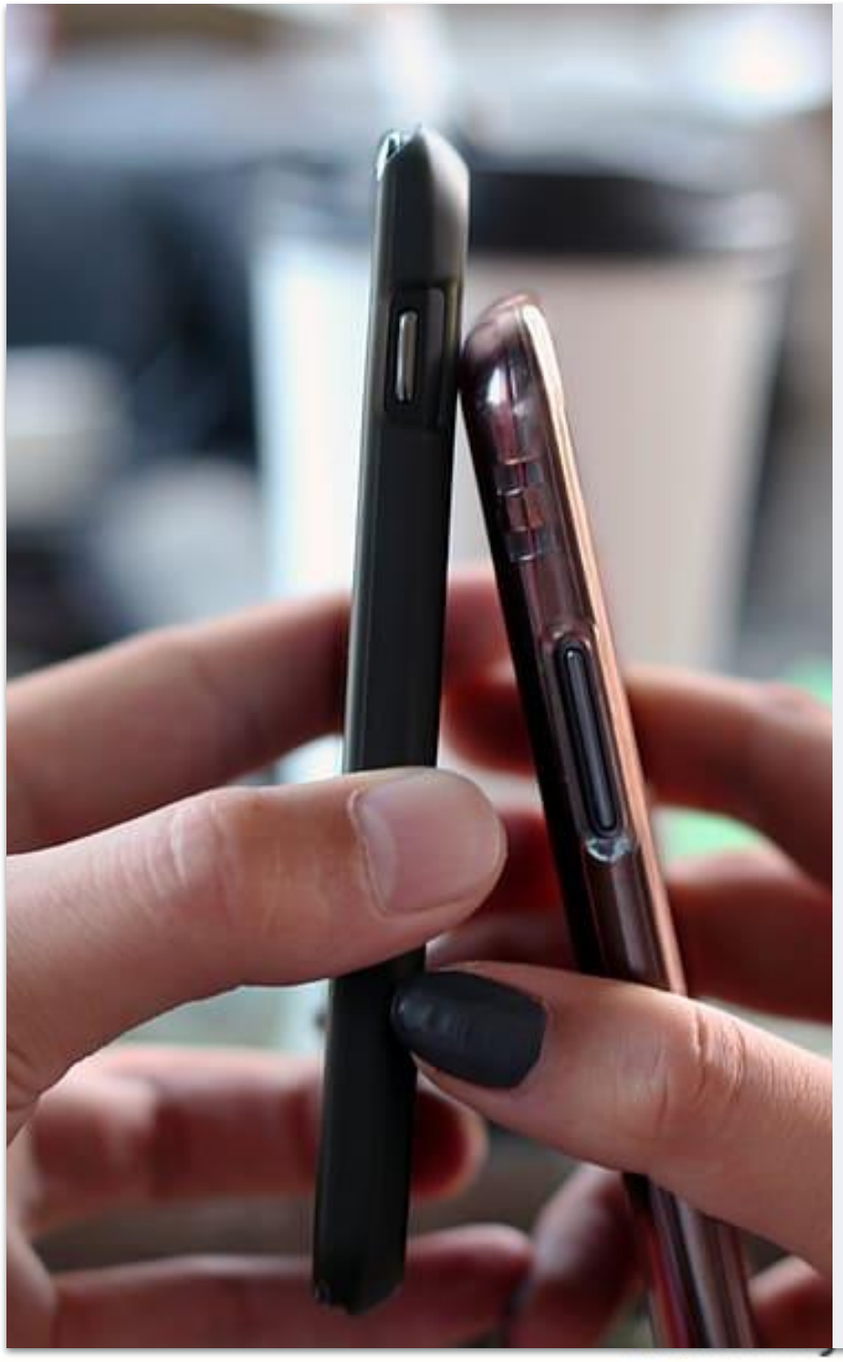
Utiliza los métodos onSaveInstanceState() y onRestoreInstanceState() para guardar y restaurar el estado de tu actividad, respectivamente. Esto es útil cuando la actividad se destruye y vuelve a crearse,



Gestión de la conectividad

¿Cómo?:

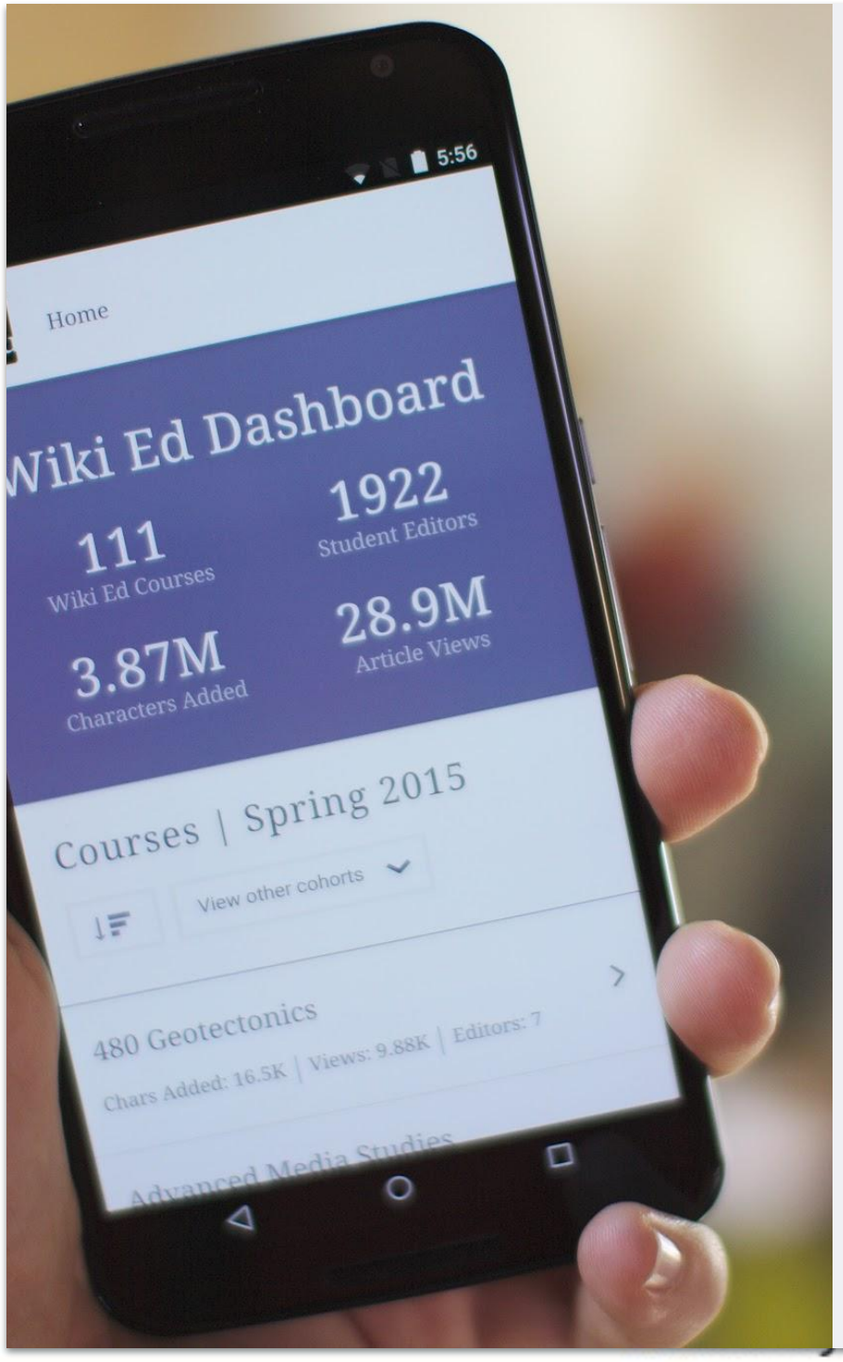
Puedes controlar el estado de la conectividad de red de la aplicación utilizando los métodos onStart() y onStop(). Por ejemplo, puedes registrar y anular el registro de receptores de difusión relacionados con la conectividad en estos métodos. Esto te permite optimizar el uso de la red y ajustar el comportamiento de la aplicación según la disponibilidad de la conexión.



Carga diferida de datos

¿Cómo?:

En lugar de cargar todos los datos en el método onCreate(), considera cargar los datos en un momento posterior del ciclo de vida, como en onStart() o onResume(). Esto puede mejorar el rendimiento de la aplicación, ya que los datos se cargarán cuando la actividad esté a punto de mostrarse o cuando ya esté visible para el usuario.



Gestión de permisos

¿Cómo?:

El ciclo de vida de una actividad se puede utilizar para solicitar permisos sensibles al usuario cuando sean necesarios. Puedes solicitar permisos en onCreate() o en onResume() y manejar la respuesta en onRequestPermissionsResult(). Esto garantiza que tu aplicación solicite los permisos en el momento adecuado y ofrece una experiencia de permisos más fluida.

Manejo de tareas en segundo plano

¿Cómo?:

Si necesitas realizar tareas en segundo plano, como descargar datos o procesar información, puedes utilizar hilos o servicios en combinación con el ciclo de vida. Por ejemplo, puedes iniciar un servicio en onStart() y detenerlo en onStop() para realizar una tarea continua en segundo plano.

¿Alguna consulta?

alkemy

› El ciclo de vida de los fragmentos

El ciclo de vida de los fragmentos

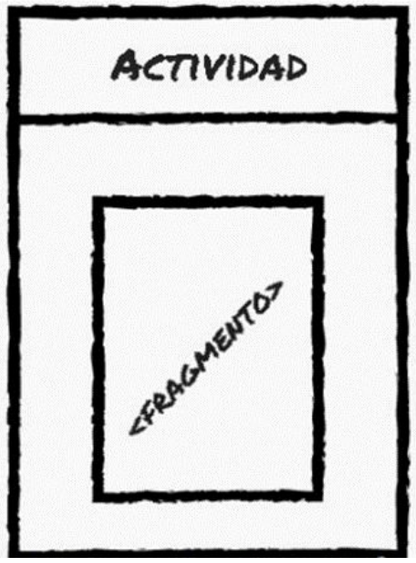
¿Qué es?:

Un Fragmento en Android es la representación de un comportamiento o una porción de interfaz de usuario.La idea detrás de este elemento es mostrar elementos de forma dinámica en la pantalla reutilizando lo que sea necesario.Viene desde la Android 3.0(Honeycomb) API 11 y se introdujo con la llegadas de las tablets y pantallas más anchas, con la necesidad de tener UI dinámicas.

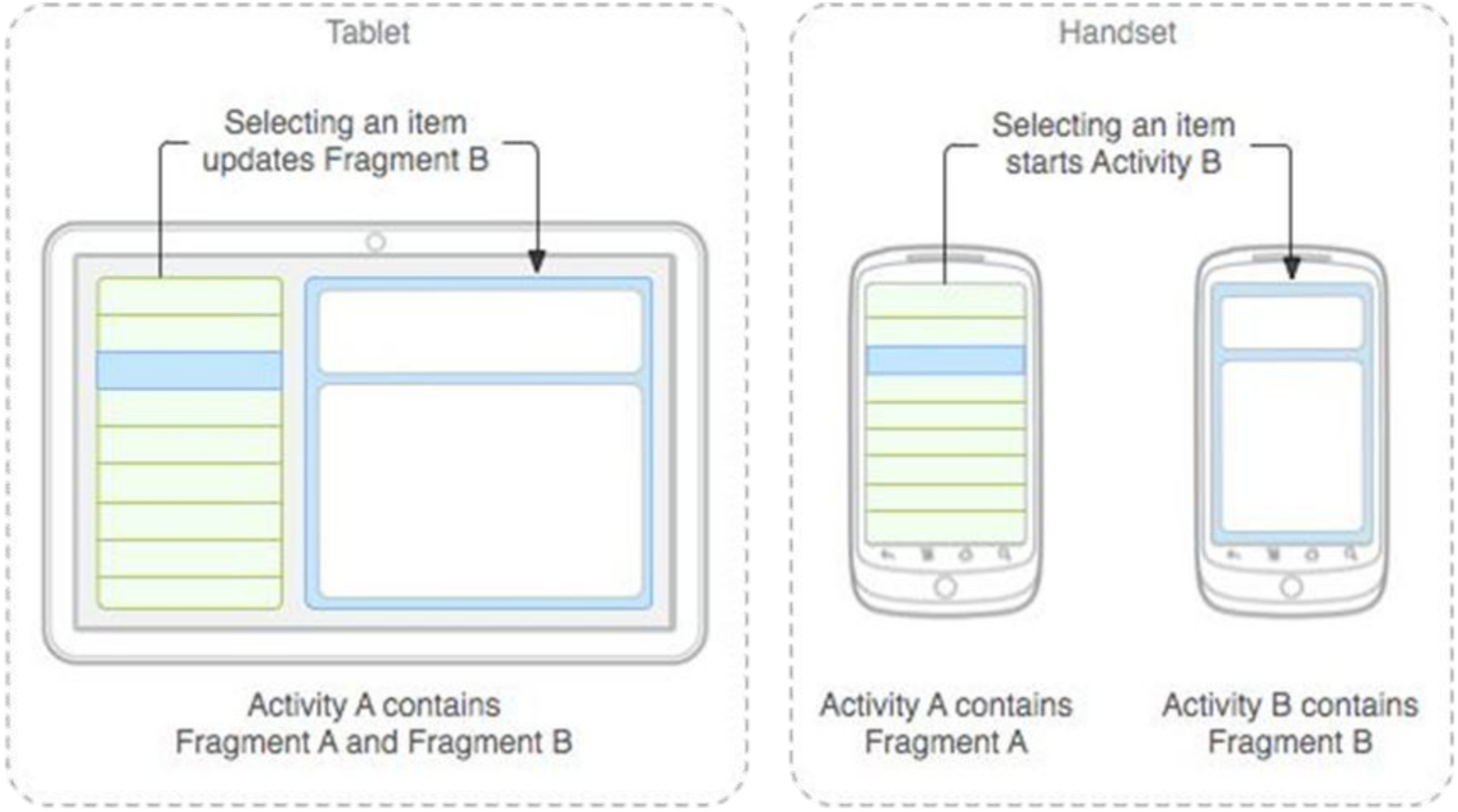
El ciclo de vida de los fragmentos

La necesidad de usar fragmentos nace con la **versión 3.0 (API 11) de Android** debido a los múltiples tamaños de pantalla que estaban apareciendo en el mercado y a la capacidad de orientación de la interfaz (**Landscape** y **Portrait**). Estas características necesitaban dotar a las aplicaciones Android de la capacidad para adaptarse y responder a la interfaz de usuario sin importar el dispositivo.

Un **fragmento** es una sección "**modular**" de interfaz de usuario embebida dentro de una **actividad anfitriona**, el cual permite versatilidad y optimización de diseño. Se trata de miniactividades contenidas dentro de una actividad anfitriona, manejando su propio diseño (un **recurso layout** propio) y **ciclo de vida**.



Fragmentos


Tablet
Selecting an item updates Fragment B
Activity A contains Fragment A and Fragment B
Handset
Selecting an item starts Activity B
Activity A contains Fragment A
Activity B contains Fragment B

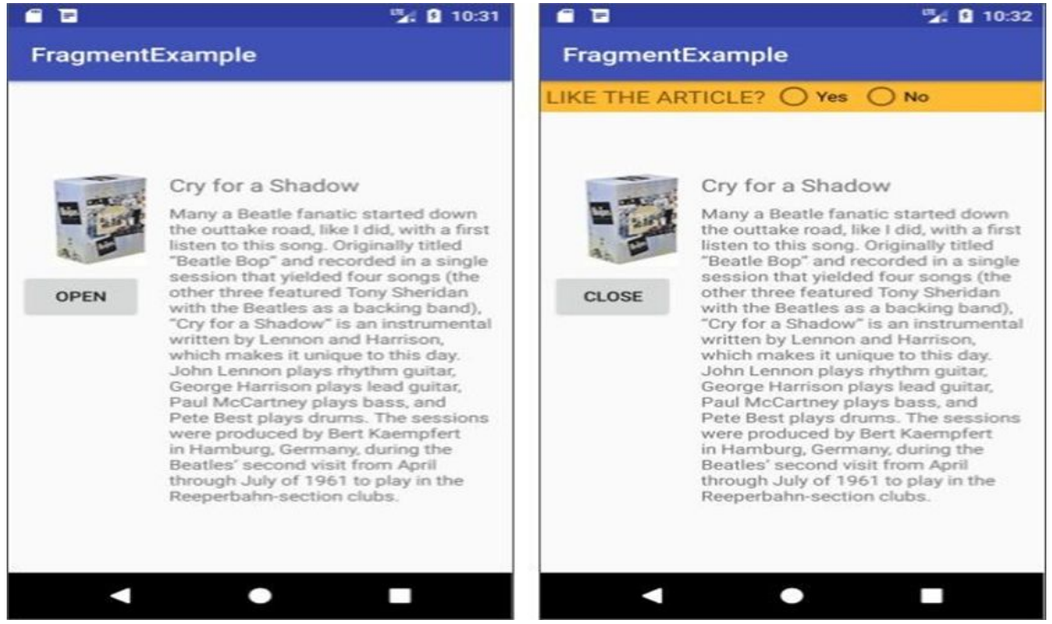
Funciones de los fragmentos

Los Fragmentos tiene otros eventos asociados a su ciclo de vida mantienen funciones como onCreate pero también agregar funciones como:
onAttach(): Este método se llama cuando el fragmento se adjunta a una actividad.
onCreateView(): En este método, se crea la vista del fragmento. Debes inflar y devolver un objeto View que represente el diseño del fragmento.
onViewCreated(): Este método se invoca después de que la vista del fragmento ha sido creada.
onDetach(): Este método se llama cuando el fragmento se desvincula de la actividad.

Funciones de los fragmentos

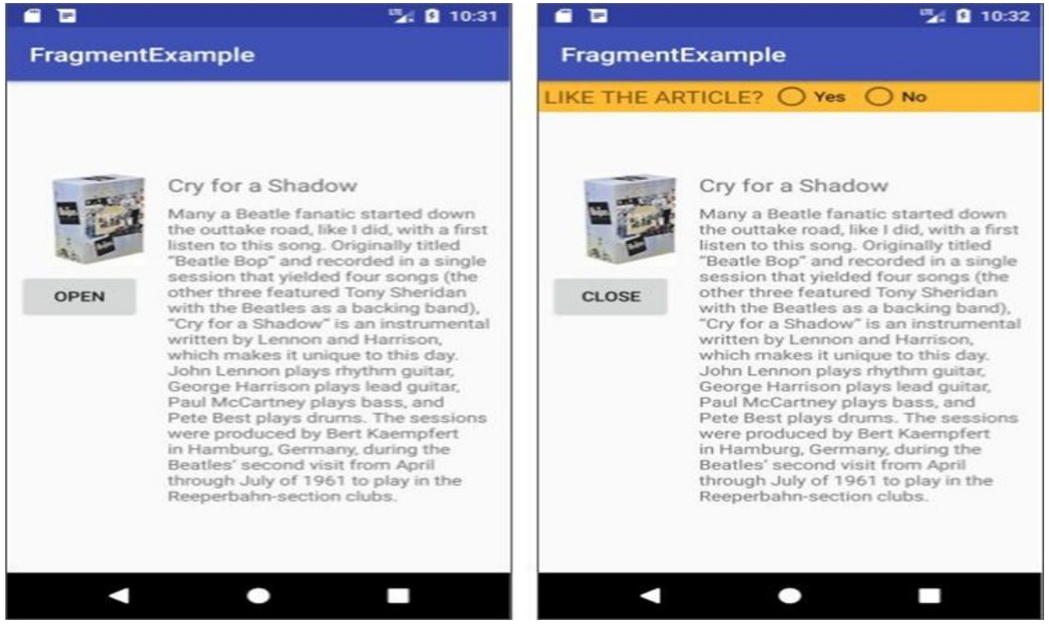
Un fragmento puede ser una parte estática de la UI de una actividad.

También puede ser una parte dinámica de UI, añadiendo o quitando su presencia, mientras la actividad está en ejecución.

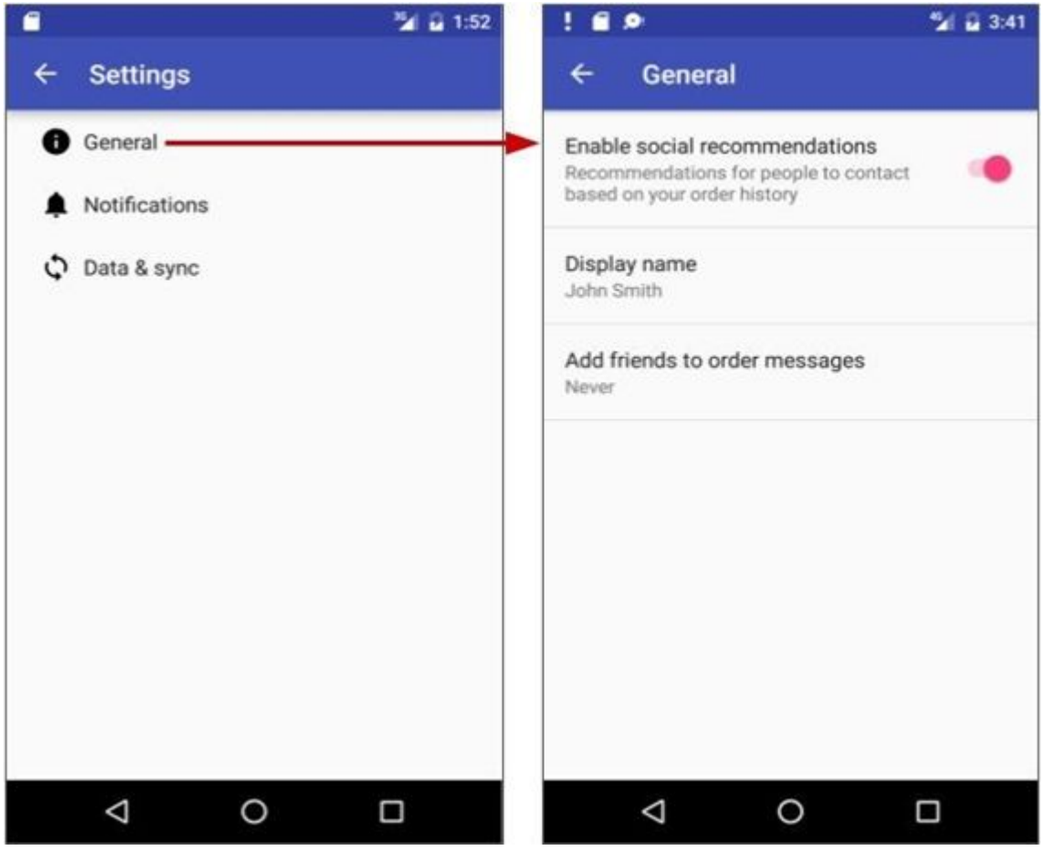


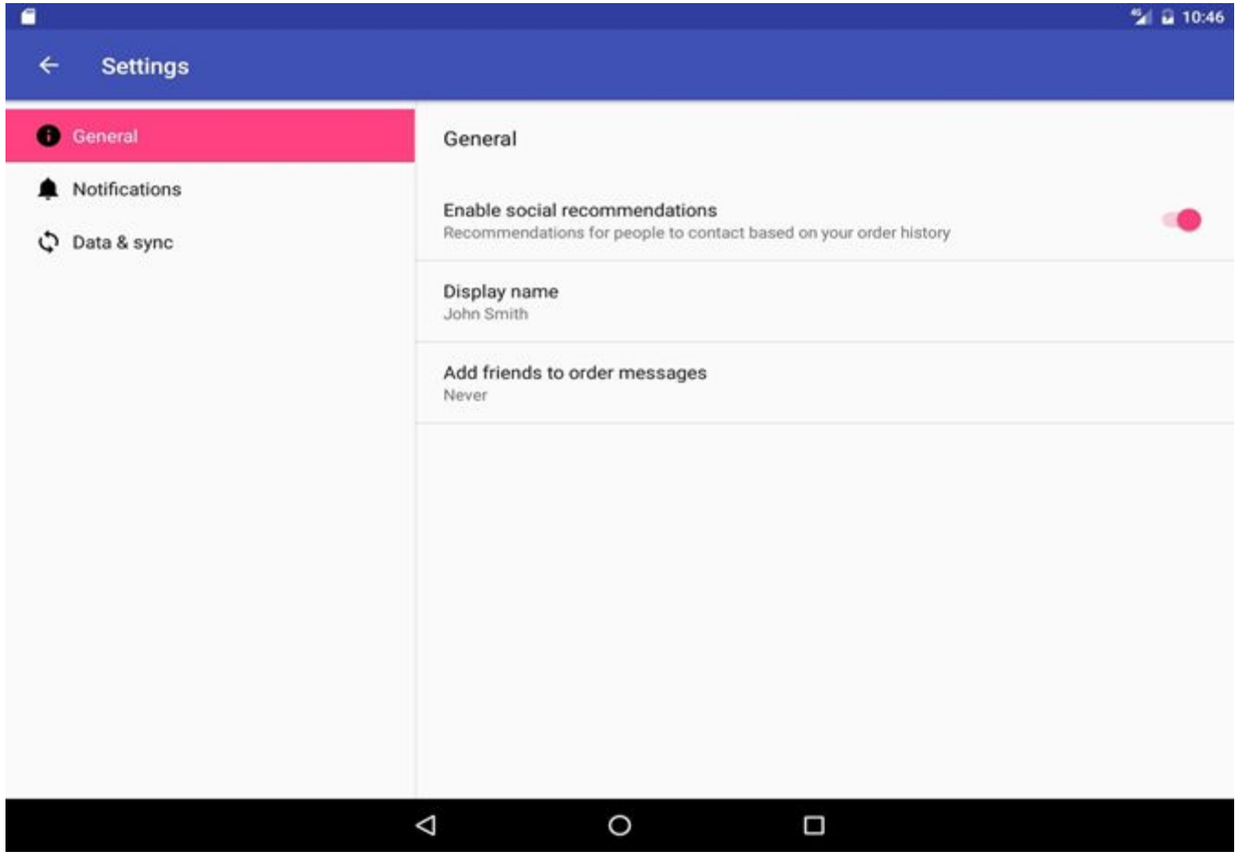
Funciones de los fragmentos

- Modularidad
  - División o segmentación del código de una actividad compleja en fragmentos para una mejor optimización y mantención.
- Reusabilidad
  - Al utilizarlos para definir comportamientos de una sección de la UI, estos pueden ser compartidos por múltiples actividades.
- Adaptabilidad
  - La representación de secciones de UI como fragmentos permite utilizar diferentes diseños según la orientación y pantalla de los dispositivos.

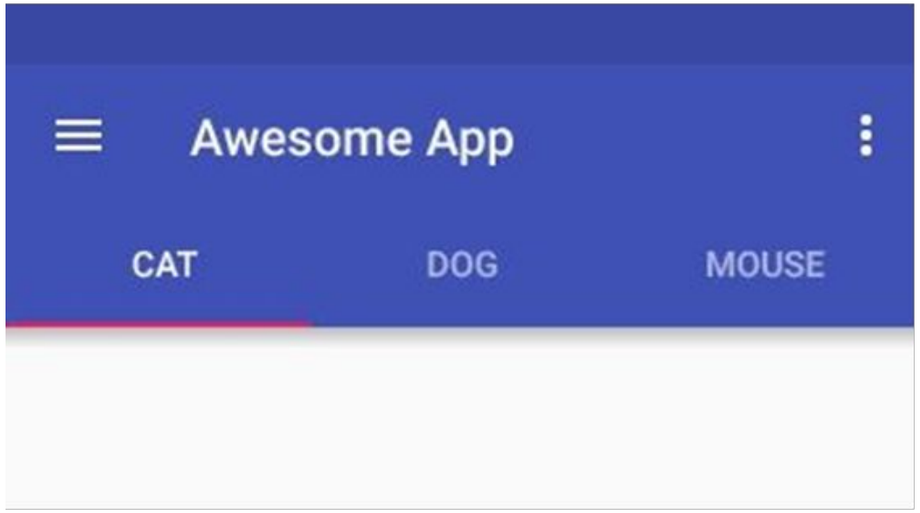


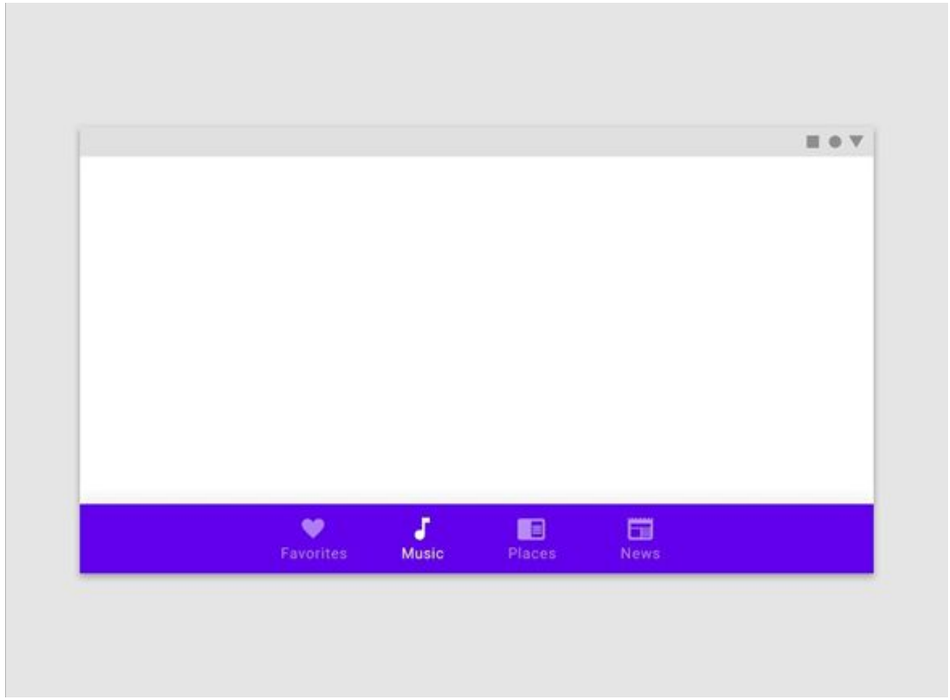
Ejemplos


1:52
Settings
General
Notifications
Data & sync
3:41
General
Enable social recommendations
Recommendations for people to contact based on your order history
Display name
John Smith
Add friends to order messages
Never

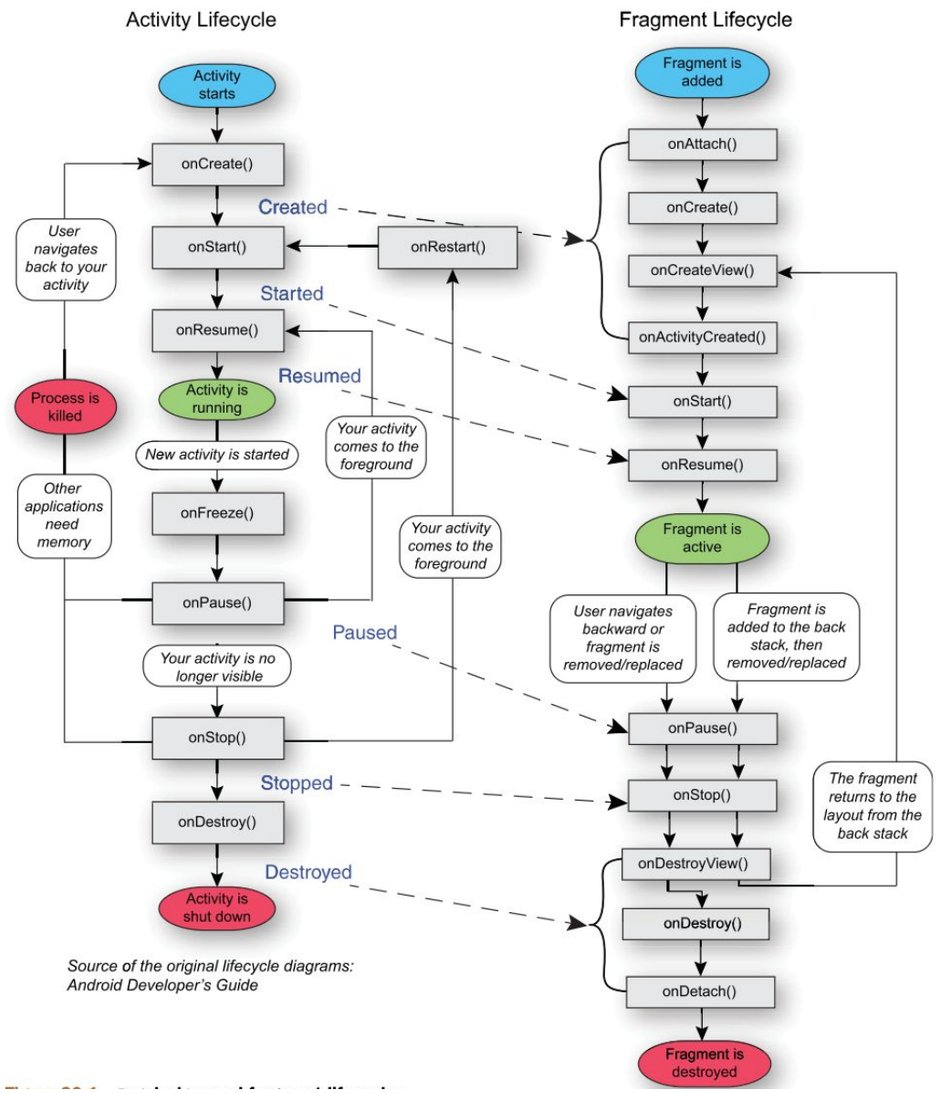

10:46
Settings
General
Notifications
Data & sync
General
Enable social recommendations
Recommendations for people to contact based on your order history
Display name
John Smith
Add friends to order messages
Never

Ejemplos





El ciclo de vida de los fragmentos



Source of the original lifecycle diagrams: Android Developer's Guide

LIVE CODING

Ejemplo en vivo

Testear el ciclo de Vida de un fragment:

Crear un Fragment y seguir el comportamiento de sus eventos

1. *Crear una actividad con fragmento incluido*
2. *Hacer override de todas las funciones del ciclo de vida*
3. *Dentro de la fun OnCreate agregar la linea*
4. *Log.i("Ciclo de vida", "<Nombre fun>")*
5. *Cambiar <Nombre fun> con nombre - correspondiente a la fun*
5. *Correr la aplicaciones*
6. *Abrir y cerrar la activity*
7. *Descripción*

Tiempo: 30 min

LIVE CODING

Ejemplo en vivo

Creando un Fragmento

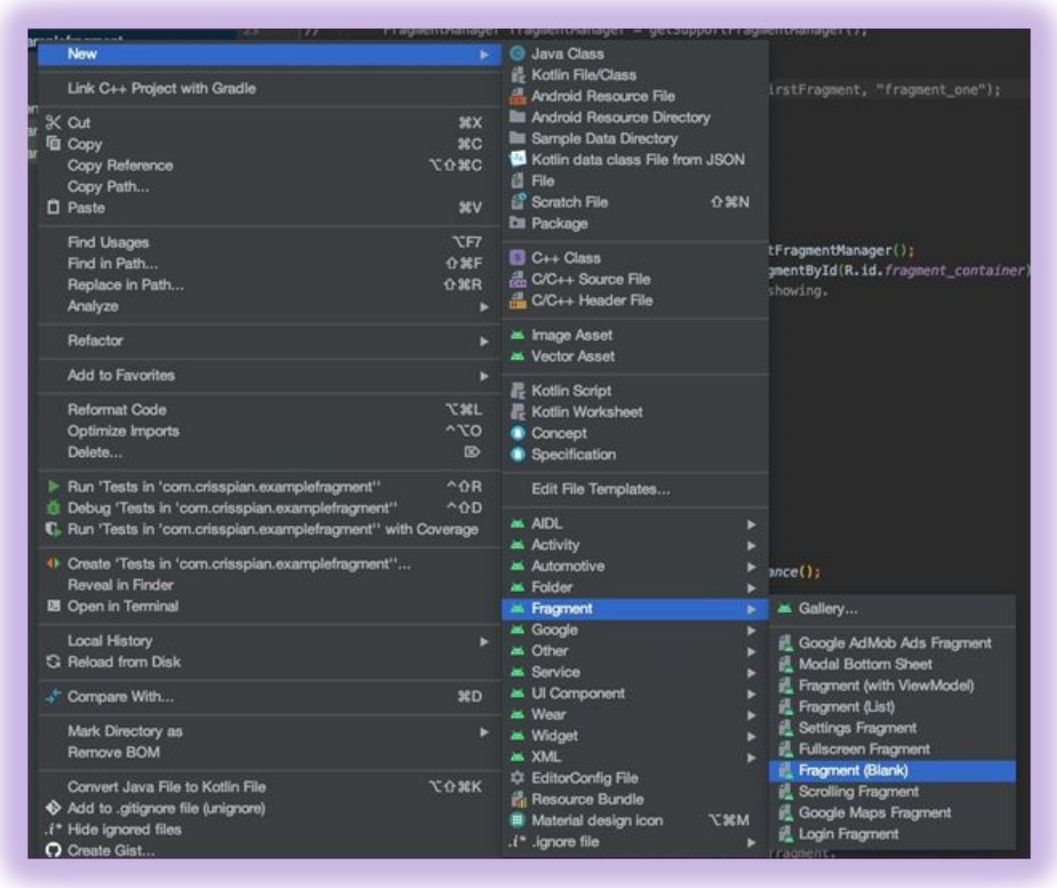
Crear un Fragment y seguir el comportamiento de sus eventos

1. Clonar el siguiente repositorio.
2. El contiene un proyecto base.
3. https://github.com/Mariocanedo/FragmentGuide0
4. ¿Que vamos a construir?

Tiempo: 45 min

Creando un nuevo Fragmento

1. Crear un nuevo fragmento.
2. Utilizar un Blank Fragment.
3. Asignarle un nombre.

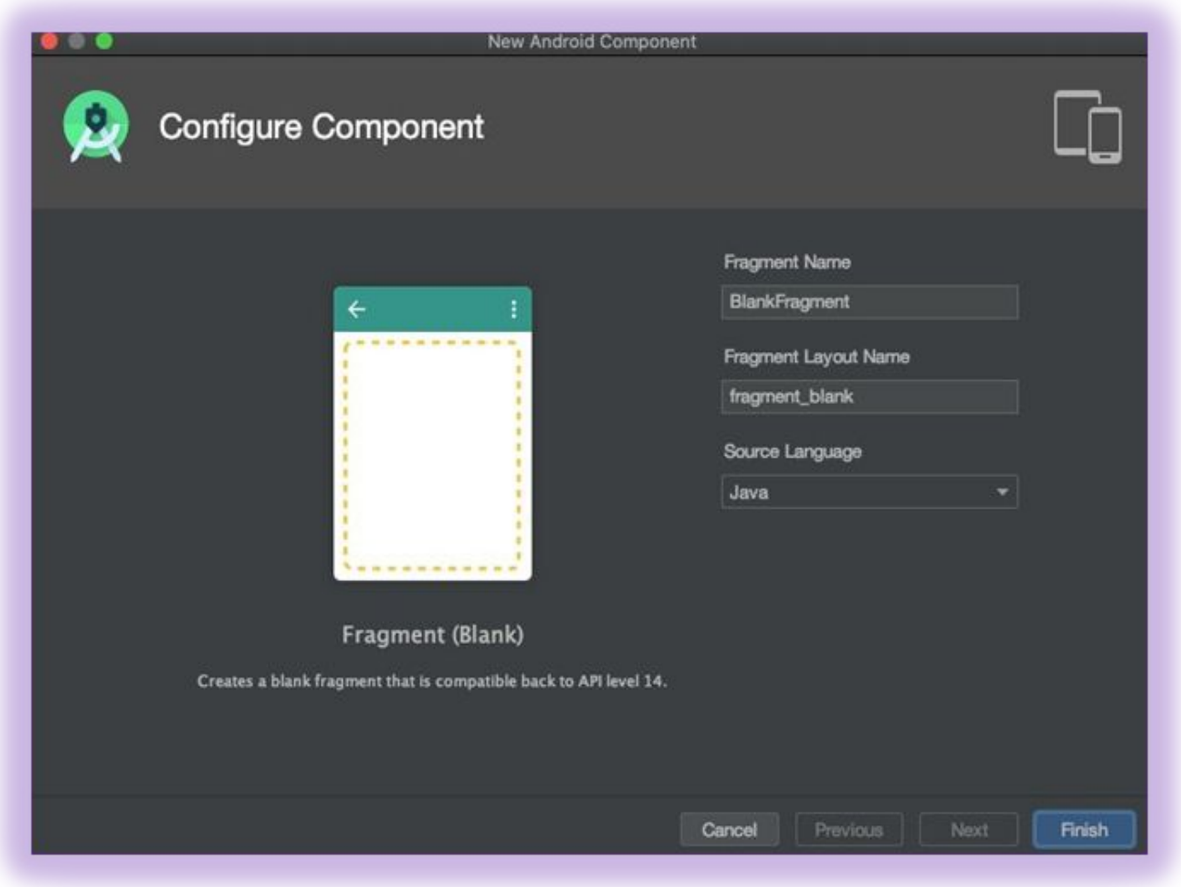


Creando un nuevo Fragmento 02

- Asignarle un nombre al fragmento
- Fijarse en la convención de

nombres para el layout.

.



Evaluando Fragmento Creado

1.El fragmento viene con algunos métodos listos, estos son llamados factory methods, esta vez los eliminaremos.

2.También eliminar las referencias a los argumentos.

3.El código del fragmento debe quedar así.

.

.

```
public class BlankFragment extends Fragment {
  public BlankFragment() {
    // Required empty public constructor
  }
@Override
public void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  }

@Override
            public View onCreateView(LayoutInflater inflater,
            ViewGroup container,  Bundle
            savedInstanceState) {
  // Inflate the layout for this fragment
  return inflater.inflate(R.layout.fragment_blank, container,
  false);
}
  }
```

› Modificando el Layout del Fragmento

Modificando Fragmento Creado 1

1.Ahora modificaremos la vista del fragmento anterior
2.Cambiar el FrameLayout por un LinearLayout y añadir lo siguiente:
3.Cambiar los valores de orientación, color de fondo y altura.

.
.

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:orientation="horizontal"
android:background="@android:color/holo_orange_dark"
android:layout_width="match_parent"
android:layout_height="wrap_content"
tools:context=".BlankFragment">
...
```

Modificando Fragmento Creado 2

1.Revisar el layout del fragmento.
2.Vamos a modificar el TextView que viene por defecto

.
.

```
<TextView
android:id="@+id/fragment_header"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:theme="@style/TextAppearance.AppCo
mpat.Headline"
android:text="@string/question_one"
android:padding="4dp"/>
```

3. Modificar el archivo strings.xml y añadir lo siguiente.

```
<string name="question_one">¿Conoces este Meme?</string>
<string name="yes_message">¡Si lo conozco!</string>
<string name="no_message">No... buuu...</string>
```

Modificando Fragmento Creado 3

1.Añadir un RadioButton

2.Añadir texto si y no en string.xml

.

.

```
<RadioGroup  android:id="@+id/radio_group"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:orientation="horizontal">
  <RadioButton  android:id="@+id/radio_button_yes"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_marginRight="8dp"
  android:text="@string/yes" />
  <RadioButton  android:id="@+id/radio_button_no"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_marginRight="8dp"
  android:text="@string/No" />
</RadioGroup>
```

Modificando Fragmento Creado 4

1. Añadir la clase para manejar el ViewBinding (recuerda activarlo en el build.gradle)

2. Añadir dos variables que manejan el estado.

```
private FragmentQuestionBinding binding;
public static final int YES = 0;
public static final int NO = 1;
```

Código del Fragmento

1.Añadir La clase para manejar el ViewBinding

2.Añadir dos variables que manejan el estado.

Añadir la lógica de selección de RadioButton

```
@Override
            public View onCreateView(LayoutInflater inflater, ViewGroup container,
            Bundle savedInstanceState) {
 binding = FragmentQuestionBinding.inflate(inflater, container, false);  final View
 view = binding.getRoot();


  binding.radioGroup.setOnCheckedChangeListener(new
  RadioGroup.OnCheckedChangeListener() {  @Override
  public void onCheckedChanged(RadioGroup group, int checkedId) {
    switch (checkedId) {
        case R.id.radio_button_yes:  binding.fragmentHeader.setText(R.string.yes_message);
        mListener.onRadioButtonChoice(YES);
        break;
        case R.id.radio_button_no:  binding.fragmentHeader.setText(R.string.no_message);
        mListener.onRadioButtonChoice(NO);
      break;  default:
        mRadioButtonChoice = NONE;
        mListener.onRadioButtonChoice(mRadioButtonChoice);  break;
    }
  }
});
```

› El método de factoría

Método factoría(factory)

¿Qué es?:

Es un patrón de diseño utilizado en el desarrollo de software, incluido el desarrollo de aplicaciones Android, para crear objetos sin especificar la clase exacta del objeto que se creará. En el contexto de Android, el método factoría se utiliza para encapsular la creación de objetos y proporcionar una forma de crear objetos de una subclase específica sin que el código cliente tenga que conocer los detalles de implementación exactos.
El método factoría es útil cuando tienes una jerarquía de clases y deseas que el código cliente interactúe con la jerarquía de una manera abstracta, sin depender de clases concretas. En Android, esto se aplica a menudo al trabajar con componentes de interfaz de usuario (UI) y fragmentos.

Los fragmentos y el método de factoría

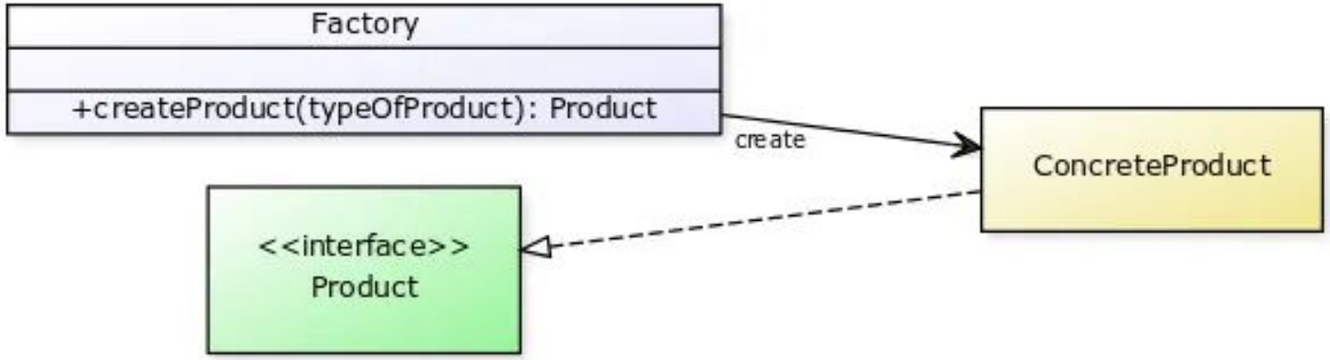
¿Cómo se usan?:

Los fragmentos se pueden crear utilizando el método de factoría, que es una práctica común para crear instancias de fragmentos con argumentos personalizados. El método de factoría es una alternativa al uso del constructor predeterminado de la clase de fragmento.

Los fragmentos y el método de factoría

¿Cómo se usan?:

Basicamente existen 3 instancias para implementar como minimo:
Clase Factory: Nos crea el producto que necesitamos.
Clase Producto Concreto: Implementa la interfaz Producto y desarrolla su método con la lógica correspondiente.
Interfaz del Producto: Contiene los métodos comunes que van a reunir los productos concretos.



LIVE CODING

Ejemplo en vivo

Implementar un Método de Factoría:

Implementar una llamada a FragmentFactory donde

1. *Crear una nueva clase "factoria"*
2. *Importar FragmentFactory a las dependencias*
3. *Hacer que la clase extienda "FragmentFactory"*
4. *hacer override de instantiate*
5. *Crear una llamada a la factory en una activity*

Tiempo: 20 min

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ Conocimos el ciclo de vida de Android
- ✓ Vimos cómo se aplica al ciclo de vida de Actividades
- ✓ Exploramos qué tenemos que tener en cuenta cuando nuestras aplicaciones van a pasar a segundo plano
- ✓ Conocimos los permisos sensibles
- ✓ Implementamos el uso de permisos
- ✓ Vimos como usar el ciclo de vida a nuestro favor
- ✓ El ciclo de vida de los Fragmentos
- ✓ Cómo se relaciona al de las actividades
- ✓ El método factoría

< ¡Muchas gracias! >

alkemy